

Mars Operations: Base Deployment

Documentazione completa di sviluppo

Corso: *Metodi di Ottimizzazione per Big Data*

Prof. Andrea Cristofari

Università di Roma “Tor Vergata” — A.A. 2025/2026

Studenti: Spiridigliozzi Davide, Rosi Federico

Descrizione del problema

Contesto e obiettivo

Il progetto *Mars Operations: Base Deployment* (MOBD) simula il posizionamento ottimale di $m = 1000$ moduli operativi $x^1, \dots, x^m$ su una mappa bidimensionale del pianeta Marte. Nell'area è già presente una postazione fissa $z^0 = (0, 0)$ (centro di controllo) e quattro stazioni:

$$z^1 = (100, 100), \quad z^2 = (-100, 100), \quad z^3 = (-100, -100), \quad z^4 = (100, -100).$$

Ogni modulo $x^i \in \mathbb{R}^2$ ha coordinate (x_1^i, x_2^i) . Il vettore delle variabili di decisione è

$$x = (x_1^1, x_2^1, x_1^2, x_2^2, \dots, x_1^{1000}, x_2^{1000})^\top \in \mathbb{R}^{2000}.$$

Si vuole **minimizzare** la funzione di costo totale:

$$\min_{x \in \mathbb{R}^{2000}} f(x) = f_1(x) + f_2(x) + f_3(x) + f_4(x).$$

Le 4 componenti di costo:

f_1 — Dispersione della rete (coesione mesh)

$$f_1(x) = 0.1 \sum_{i < j} \left(1 - e^{-\|x^i - x^j\|^2} \right)$$

Misura la *perdita di coesione* della rete mesh. Per ogni coppia (i, j) , il termine $1 - e^{-\|x^i - x^j\|^2}$ è vicino a 0 quando i moduli sono vicini, e tende a 1 quando si allontanano. Si sommano tutte le $\binom{1000}{2} = 499\,500$ coppie.

f_2 — Distanza dalle stazioni fisse

$$f_2(x) = \frac{1}{m} \sum_{i=1}^m \sum_{q=0}^4 \|x^i - z^q\|^2$$

Penalità quadratica sull'allontanamento dalle $Q = 5$ stazioni fisse.

f_3 — Deviazione dalla distanza operativa ideale

$$f_3(x) = 1000 \sum_{i=1}^m \left(\frac{\log(1 + \|x^i - s^i\|^2)}{\log(1 + r^2)} - 1 \right)^2, \quad r = 100$$

dove $s^i = ((-1)^i \cdot i \cdot 0.2, (-1)^i \cdot i \cdot 0.2)$ è il punto di riferimento operativo del modulo i . La funzione è minimizzata (vale 0) esattamente quando $\|x^i - s^i\| = r = 100$ per ogni i .

f_4 — Costo ambientale black-box

f_4 modella fenomeni non ideali (ombreggiamento radio, micro-fratture del regolite, polvere, interferenze elettromagnetiche, congestione locale). **Non ha espressione analitica nota:** viene valutata esclusivamente tramite il simulatore esterno `mobd` con l'opzione `-b`. Il simulatore è deterministico e richiede in input il file `x.txt` (2000 valori reali, un valore per riga).

Ottimizzazione non vincolata e condizioni di ottimalità

Il problema è un'ottimizzazione *non vincolata* in $\mathbb{R}^n$ con $n = 2000$:

$$\min_{x \in \mathbb{R}^n} f(x).$$

Condizioni necessarie del primo ordine

Se x^* è un minimo locale di f e f è differenziabile in x^* , allora:

$$\nabla f(x^*) = 0.$$

Un punto che soddisfa questa condizione è detto *punto stazionario*.

Condizioni necessarie del secondo ordine

Se x^* è un minimo locale di f e f è due volte differenziabile in x^* , allora:

$$\nabla f(x^*) = 0 \text{ e } \nabla^2 f(x^*) \succeq 0.$$

Condizioni sufficienti del secondo ordine

Se $\nabla f(x^*) = 0$ e $\nabla^2 f(x^*) \succ 0$, allora x^* è un minimo locale stretto.

Metodi a discesa: direzioni gradient-related

Un metodo iterativo $x_{k+1} = x_k + \alpha_k d_k$ garantisce la discesa se la direzione d_k è *gradient-related*, cioè soddisfa:

$$\nabla f(x_k)^\top d_k \leq -c_1 \|\nabla f(x_k)\|^2, \quad \|d_k\| \leq c_2 \|\nabla f(x_k)\|,$$

per costanti $c_1, c_2 > 0$. La direzione del **gradiente negativo** $d_k = -\nabla f(x_k)$ soddisfa entrambe le condizioni con $c_1 = c_2 = 1$.

Block Coordinate Descent (BCD)

Partizionamento in blocchi

Si partiziona il vettore $x \in \mathbb{R}^n$ in m blocchi:

$$x = (x^1, x^2, \dots, x^m), \quad x^i \in \mathbb{R}^{n_i}, \quad \sum_{i=1}^m n_i = n.$$

Nel nostro caso: $m = 1000$ blocchi, $n_i = 2$ per ogni i (le coordinate 2D del modulo i), $n = 2000$.

BCD Gauss-Seidel (variante ciclica)

Ad ogni iterazione, si aggiorna un solo blocco i alla volta, tenendo fissi tutti gli altri. L'aggiornamento del blocco i all'epoca k è:

$$x_{k+1}^i = x_k^i - \alpha_k \nabla_{x^i} f(x_k),$$

dove $\nabla_{x^i} f$ è il *gradiente parziale* rispetto al blocco i . Nella variante **Gauss-Seidel**, i blocchi già aggiornati nell'epoca corrente vengono usati immediatamente per il calcolo dei gradienti successivi (in contrast con la variante Jacobi che usa sempre le posizioni all'inizio dell'epoca).

Randomized BCD (RBCD) — Gauss-Seidel con permutazione casuale

RBCD Gauss-Seidel

Ad ogni epoca k :

1. Si genera una permutazione casuale σ_k di $\{1, \dots, m\}$.
2. Si aggiornano i blocchi nell'ordine $\sigma_k(1), \sigma_k(2), \dots, \sigma_k(m)$.
3. Ogni aggiornamento usa le posizioni **più recenti** disponibili.

L'uso di una permutazione casuale (invece di un ordine fisso) migliora le proprietà di convergenza e riduce il rischio di comportamenti patologici legati alla struttura del problema.

Stepsize decrescente (Diminishing Stepsize)

Schema di stepsize decrescente

Lo stepsize al passo k è definito come:

$$\alpha_k = \frac{\alpha_0}{(1 + \beta \cdot k)^\gamma},$$

dove $\alpha_0 > 0$ è lo stepsize iniziale, $\beta > 0$ controlla la velocità di decrescita, e $\gamma \in (0, 1]$ è l'esponente di decrescita.

Condizioni sufficienti per la convergenza (Teorema di convergenza)

Una sequenza di stepsize $\{\alpha_k\}$ garantisce la convergenza se:

$$\lim_{k \rightarrow \infty} \alpha_k = 0 \text{ e } \sum_{k=0}^{\infty} \alpha_k = +\infty.$$

Per il nostro schema $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$:

- $\lim_{k \rightarrow \infty} \alpha_k = 0$ è verificata per ogni $\gamma > 0$.
- $\sum_k \alpha_k = +\infty$ è verificata se e solo se $\gamma \leq 1$.

Con $\gamma = 0.6 \leq 1$, entrambe le condizioni sono soddisfatte.

Teorema di convergenza RBCD — Gauss-Seidel

Sia $f: \mathbb{R}^n \rightarrow \mathbb{R}$ continuamente differenziabile con gradiente Lipschitz. Sia $\{x_k\}$ la sequenza generata dall'algoritmo RBCD Gauss-Seidel con stepsize decrescente che soddisfa $\alpha_k \rightarrow 0$ e $\sum_k \alpha_k = +\infty$. Allora ogni punto di accumulazione della sequenza $\{x_k\}$ è un **punto stazionario** di f , cioè soddisfa $\nabla f(x^*) = 0$.

Osservazione 1. Nel caso non convesso, la complessità per ottenere $\|\nabla f(x_k)\| \leq \varepsilon$ è $O(\varepsilon^{-2})$ iterazioni, analoga a quella del metodo del gradiente classico.

Differenze finite in avanti per il gradiente di f_4

Poiché f_4 è una black-box senza espressione analitica, si approssima il suo gradiente parziale tramite **differenze finite in avanti** (forward finite differences):

$$\frac{\partial f_4}{\partial x_j^i}(x) \approx \frac{f_4(x + H \cdot e_{2i+j}) - f_4(x)}{H}, j \in \{0, 1\},$$

dove $H > 0$ è il passo di perturbazione (nel codice $H = 10^{-4}$) ed e_k è il k -esimo vettore della base canonica di $\mathbb{R}^{2000}$.

Per $m = 1000$ moduli con 2 coordinate ciascuno, occorrono $2m = 2000$ chiamate al simulatore per epoca (più 1 per il valore base $f_4(x)$).

Scelta di H

Un valore di H troppo grande introduce errore di troncamento, uno troppo piccolo causa cancellazione numerica. Il valore $H = 10^{-4}$ rappresenta un buon compromesso per le scale del problema (coordinate nell'ordine di 10^1 – 10^2 km).

Limiti iniziali e soluzioni adottate

Problema 1: costo computazionale di f_4

Limite

Ogni valutazione del simulatore richiede circa 5 secondi. Con 2000 chiamate per epoca in modo sequenziale: $2000 \times 5 \text{ s} \approx 2.78$ ore per epoca. Con 20 epoche: ≈ 55.5 ore. Assolutamente inattuabile.

Soluzione: parallelizzazione con `ThreadPoolExecutor`.

Le 2000 perturbazioni sono indipendenti (ogni perturbazione riguarda un diverso indice del vettore x , tenendo fissi tutti gli altri). Si possono quindi eseguire *in parallelo* su più thread. Con N workers, il tempo si riduce a $\lceil 2000/N \rceil \times 5 \text{ s}$. Su una macchina con 32 CPU: $\lceil 2000/32 \rceil \times 5 \text{ s} = 63 \times 5 = 315 \text{ s} \approx 5.25 \text{ min/epoca}$.

Problema 2: coerenza della base per le differenze finite

Limite

Nella variante Gauss-Seidel, i blocchi vengono aggiornati in sequenza durante un'epoca. Se si calcolasse il gradiente di f_4 *on-the-fly* (mentre x cambia), il valore base $f_4(x)$ usato nelle differenze finite sarebbe inconsistente con il valore di x in quel momento.

Soluzione: strategia “stale-base”.

All'inizio di ogni epoca si scatta uno *snapshot* $\hat{x}$ dello stato corrente e si calcola $\hat{f}_4 = f_4(\hat{x})$. Tutte le 2000 perturbazioni usano la stessa base $\hat{x}$, garantendo che il denominatore delle differenze finite sia sempre il medesimo punto di partenza. Questo introduce una leggera staleness del gradiente di f_4 rispetto allo stato x a fine epoca, ma è accettabile poiché lo stepsize è piccolo.

Problema 3: inizializzazione e trappole locali

Limite

Un'inizializzazione casuale (ad esempio $x = 0$ o posizioni uniformi) porta tipicamente a valori elevati di f_3 , il che crea un gradiente dominante che può portare l'algoritmo lontano da buone soluzioni per i componenti f_1, f_2, f_4 .

Soluzione: warm start con $f_3 = 0$ per costruzione.

Si costruisce un'inizializzazione in cui ogni modulo è posizionato esattamente a distanza $r = 100$ dal suo punto di riferimento s^i , in modo che $f_3 = 0$ al punto iniziale. La costruzione è:

$$x_0^i = s^i + r \cdot \frac{-s^i}{\|s^i\|},$$

cioè si parte da s^i e ci si sposta di r nella direzione opposta a s^i (verso l'origine). Si verifica che $\|x_0^i - s^i\| = r$ quindi $\phi_i = 0$ e $f_3 = 0$.

Problema 4: I/O intensivo su disco

Limite

Ogni chiamata al simulatore richiede di scrivere un file da 2000 righe su disco. Con 2000+ scritture per epoca, l'I/O su disco potrebbe diventare collo di bottiglia.

Soluzione: RAM disk (/dev/shm).

Tutti i file temporanei vengono scritti su `/dev/shm`, che è un filesystem in memoria RAM su Linux. Le scritture sono ordini di grandezza più veloci rispetto al disco fisico.

Problema 5: gradiente di f_1 di complessità $O(m^2)$

Limite

Il gradiente di f_1 richiede la somma su tutti gli $m - 1 = 999$ moduli per ogni blocco i . Con $m = 1000$ blocchi per epoca, la complessità totale è $O(m^2)$ per f_1 .

Soluzione: calcolo vettorizzato con NumPy.

Usando operazioni NumPy (broadcasting e `einsum`), il calcolo è eseguito in C sotto il cofano, evitando cicli Python espliciti. Questo riduce drasticamente i tempi pur mantenendo complessità $O(m^2)$.

Riepilogo delle scelte progettuali

Aspetto	Problema/Vincolo	Soluzione adottata
Algoritmo	Dimensione $n = 2000$, no struttura convessa garantita	RBCD Gauss-Seidel, convergenza a punti stazionari
Gradiente f_4	Black-box, no formula analitica	Differenze finite in avanti, $H = 10^{-4}$
Costo computazionale	2000 chiamate/epoca $\times 5s = 2.8h$ sequenziale	ThreadPoolExecutor con N worker paralleli
I/O	Scritture frequenti su disco $\rightarrow$ collo di bottiglia	RAM disk <code>/dev/shm</code>
Inizializzazione	Rischio trappole locali con $f_3 \gg 0$	Warm start costruttivo con $f_3 = 0$
Coerenza FD	Base inconsistente durante l'epoch Gauss-Seidel	Strategia stale-base: snapshot a inizio epoca
Robustezza	Interruzioni durante esecuzioni lunghe	Checkpointing: salva <code>best_X</code> dopo ogni epoca
Stepsize	Garantire convergenza teorica	Schedule decrescente $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$, $\gamma = 0.6$

Come eseguire il codice

```
# Modalita' FULL (spec-compliant, raccomandata per la competizione):  
python mobd_optimizer.py --mode full --full-epochs 20
```

```
# Modalita' FAST (solo gradienti analitici, per debug):  
python mobd_optimizer.py --mode fast --fast-epochs 100
```

```
# Modalita' BOTH (warm-up analitico poi full):  
python mobd_optimizer.py --mode both --fast-epochs 50 --full-epochs 20
```

```
# Riprendere da x.txt esistente (default) o ricominciare dal warm start:  
python mobd_optimizer.py --mode full --no-resume
```

Controllare la soluzione con il simulatore:

.linux/mobd x.txt -b # solo f_4

.linux/mobd x.txt -t # tutto (solo per verifica manuale, non nel codice)

Parametri disponibili:

- `--alpha0`: stepsize iniziale (default 0.01)
- `--beta`: parametro β dello schedule (default 0.05)
- `--gamma`: esponente γ dello schedule (default 0.60)
- `--seed`: seme random (default 42)
- `--workers`: numero di thread per f_4 (default: `os.cpu_count()`)
- `--no-resume`: ignora `x.txt` esistente, riparte dal warm start